
ICCAD 2026 CAD Contest Problem D

Timing Fixing by Useful Skew

Aaron Wu

Department of Electrical Engineering
National Taiwan University
b13901011@ntu.edu.tw

Yucheng Liu

Department of Electrical Engineering
National Taiwan University
b13901104@ntu.edu.tw

Sheng-Yu Cheng

Department of Electrical Engineering
National Taiwan University
b13901088@ntu.edu.tw

Hong-Bin Lai

Department of Electrical Engineering
National Taiwan University
b13901078@ntu.edu.tw

Abstract

1 Introduction

Timing closure is a critical task in modern VLSI design. As clock frequency increases, setup and hold violations become harder to fix without disturbing the functional logic of the circuit. ICCAD Contest 2026 Problem D focuses on this setting: given an initial clock tree, a buffer library, and data-path delay reports, the goal is to improve timing quality by modifying the clock tree while preserving the original data-path logic.

The key idea behind this problem is useful skew. Instead of treating all clock skew as harmful, useful-skew optimization intentionally changes clock arrival times at flip-flops to improve timing margins. For a setup-critical path, delaying the capture clock can provide more time for data propagation. For a hold-critical path, delaying the launch clock can postpone data launch and reduce the risk of early data arrival. Therefore, by inserting or resizing buffers in the clock tree, the optimizer can improve setup and hold slack without changing combinational logic.

However, this optimization is highly coupled. A clock-tree edit does not affect only one data path; it changes the clock arrival time of an entire downstream subtree. As a result, a move that improves one violating path may degrade other paths that share the same clock-tree nodes. In addition, buffer insertion and upsizing may increase area, while aggressive area recovery may undo previous timing improvements. The objective is therefore not simply to fix the worst violation, but to balance SS setup improvement, FF hold improvement, and clock-tree area.

In this work, we formulate timing fixing by useful skew as a constrained local-search problem over legal clock-tree states. Our solver maintains a mutable clock-tree representation, evaluates timing and area metrics after local edits, and uses reversible trial operations to explore the search space efficiently. Candidate-generation policies decide which clock-tree edits should be evaluated, while acceptance strategies decide how the solver moves from one tree state to another. The experimental discussion focuses on how these policies trade off setup slack, hold slack, and area.

The rest of this report is organized as follows. Section 2 formulates the timing model, objective, and legal action space. Section 3 describes the local-search framework, including candidate generation, action evaluation, acceptance strategies, and legality checks. Section 4 reports experimental results and discusses the trade-offs among the tested optimizers. Section 5 concludes the report and outlines future improvements.

2 Problem Formulation

2.1 Input Design and Legal Solution Space

Each testcase consists of an initial clock tree, a buffer library, and two data-path delay reports under the SS and FF corners. The clock tree contains a clock source as the root, clock buffers as internal nodes, and flip-flops as leaf nodes. The buffer library specifies the available buffer types, including their area, maximum fanout constraint, and fanout-dependent delay under each timing corner. The delay reports give the clock period and the fixed data-path delay between launch and capture flip-flops.

The optimizer is allowed to improve timing only by modifying the clock tree. The data-path delay is fixed, so the combinational logic is not part of the optimization space. A legal output must preserve all original contest nodes and their names, and the relative order among original nodes in the clock tree must remain unchanged. Newly inserted buffers must have unique names, all pins must be connected, each flip-flop must have a single driver, and every buffer must satisfy the fanout limit specified by the buffer library.

We model a solution as a clock-tree state. A state records the cell type of each buffer, the parent-child connectivity of the clock tree, and the set of buffers inserted by the optimizer. The legal solution space consists of all clock trees reachable from the input tree by legal clock-tree edits while satisfying the structural constraints above.

2.2 Clock Arrival Time and Useful Skew

For each flip-flop, the clock arrival time is the total delay from the clock source to that flip-flop through the clock tree. Since buffer delay depends on the selected cell type, the fanout count, and the timing corner, the clock arrival time is also corner-dependent.

A data path connects a launch flip-flop to a capture flip-flop through combinational logic. Let D_{clk}^{launch} and $D_{clk}^{capture}$ denote the clock arrival times from the clock source to the launch and capture flip-flops. We define clock skew as

$$\text{Skew} = D_{clk}^{capture} - D_{clk}^{launch}. \quad (1)$$

With this convention, positive skew means that the capture clock arrives later than the launch clock. A clock-tree edit may change the arrival time of an entire downstream subtree, so the skew of many data paths can change after a single local modification.

2.3 Setup/Hold Slack and Timing Metrics

Let D_{data} be the data-path delay, T_{clk} be the clock period, T_{setup} be the setup requirement, and T_{hold} be the hold requirement. With the skew convention above, the setup and hold slacks are

$$\text{Slack}_{setup} = T_{clk} - T_{setup} - D_{data} + \text{Skew}, \quad (2)$$

$$\text{Slack}_{hold} = D_{data} - T_{hold} - \text{Skew}. \quad (3)$$

A negative setup slack means that data arrives too late at the capture flip-flop. A negative hold slack means that data arrives too early and may overwrite the previous value. Therefore, a setup violation can often be improved by increasing the capture clock delay, while a hold violation can often be improved by increasing the launch clock delay.

Timing quality is measured using total negative slack (TNS) and worst negative slack (WNS). WNS captures the most severe timing violation among all paths, while TNS measures the accumulated amount of negative slack. In this problem, setup timing is evaluated under the setup-oriented SS corner, and hold timing is evaluated under the hold-oriented FF corner.

2.4 Timing-Area Objective and Internal Proxy Score

The objective combines timing and area. Timing quality is measured by TNS and WNS under the SS and FF corners, while area is the total area of the clock-tree buffers. Given a baseline metric value x^{ori} and a final value x , we use the relative improvement form

$$I(x, x^{ori}) = 1 - \frac{x}{x^{ori}}, \quad (4)$$

when x^{ori} is nonzero. This form rewards reducing negative timing metrics toward zero and rewards reducing area below the baseline.

The official contest score is a weighted sum of setup improvement, hold improvement, and area reduction:

$$\begin{aligned} \text{Score} = & \alpha \left[\left(1 - \frac{\text{TNS}_{SS}}{\text{TNS}_{SS}^{ori}} \right) + \left(1 - \frac{\text{WNS}_{SS}}{\text{WNS}_{SS}^{ori}} \right) \right] + \beta \left[\left(1 - \frac{\text{TNS}_{FF}}{\text{TNS}_{FF}^{ori}} \right) + \left(1 - \frac{\text{WNS}_{FF}}{\text{WNS}_{FF}^{ori}} \right) \right] \\ & + \gamma \left(1 - \frac{\text{Area}}{\text{Area}^{ori}} \right). \end{aligned} \quad (5)$$

The official weights are not disclosed, so the optimizer cannot directly use the official score as its search objective. According to the Q&A session, the organizers indicated that the weights roughly satisfy $\alpha > \beta \approx \gamma$. Based on this hint, we use a proxy score with fixed weights $\alpha = 0.5$, $\beta = 0.25$, and $\gamma = 0.25$ to guide move selection during optimization. These parameters are not claimed to be the hidden official weights; they are used only as an internal ranking function that reflects the relative importance of setup repair, hold repair, and area control.

This objective captures the main trade-off in the problem. Aggressive buffer insertion or upsizing may improve setup or hold slack, but the same move can reduce the area component and may also hurt timing on other paths. On the other hand, area recovery may remove unnecessary delay but can also undo a previous timing repair. Therefore, the optimizer must balance timing improvement and area cost rather than optimizing only one metric independently.

2.5 Local Edit Semantics

Our optimizer explores the solution space using three local edit primitives during search. First, an *insert* move places a new buffer on an existing clock-tree edge. This increases the clock delay of the downstream subtree and can be used to adjust useful skew, but it also increases area.

Second, a *resize* move changes the cell type of an existing buffer. Resizing gives a finer way to increase or decrease clock delay and area without changing the clock-tree topology. Depending on the selected cell type, a resize move may either improve timing by adding delay or reduce area by using a smaller buffer.

Third, a *remove* move deletes a buffer that was inserted by the optimizer in an earlier step and reconnects the surrounding tree. This move is used only as an internal search operation for reversible exploration and area recovery. Original contest nodes remain part of the design and are never removed. Thus, the final output still satisfies the legal modification constraints of the problem.

Because the number of possible edit sequences is large, the solver cannot enumerate the full state space. Instead, each iteration generates a smaller set of candidate moves and evaluates their impact on the current timing and proxy score. The rest of the report describes how we construct these candidate sets and how different search policies decide which move to accept.

3 Methodology

3.1 Solver Overview

After defining the legal edit operations and the timing-area objective, the remaining question is how to search the large space of legal clock-tree states efficiently. Our solver uses a local-search framework that repeatedly moves from the current clock tree to a nearby legal state. Each iteration is organized around two decisions: which candidate moves should be evaluated, and which evaluated move should be accepted as the next state.

We separate these two decisions into a **candidate-generation policy** and an **acceptance strategy**. The candidate-generation policy defines the local neighborhood visible to the solver in the current iteration. It may focus on violating timing paths, expand around upstream clock-tree regions, or include remove and resize moves for area recovery. The acceptance strategy then decides how the solver moves inside this neighborhood. It may greedily accept the best immediate improvement, probabilistically accept a temporary degradation, or use short-term memory to avoid cycling.

Algorithm 1 summarizes the overall optimization flow. Each iteration first constructs a bounded candidate set, evaluates the candidates through temporary application, and then applies one move according to the acceptance strategy. The solver also maintains the best-seen state separately from the current state, because exploratory strategies may temporarily move to lower-scoring states.

The rest of this section follows the structure of Algorithm 1. Section 3.2 describes the optimization state and the incremental timing evaluation used inside the temporary apply-score-undo loop. Section 3.3 describes how candidate-generation policies construct the move set M . Section 3.4 describes how acceptance strategies choose the move m^* from the evaluated set. Finally, Section 3.5 explains how the best-seen internal state is converted into a legal output file.

3.2 Optimization State and Incremental Evaluation

The optimizer maintains three main data structures during search. The `ClockTree` is the mutable solution state and stores the active topology and buffer cell types. The `DataPathGraph` stores the fixed timing paths, including the launch flip-flop, capture flip-flop, and data delay of each path. Since useful-skew optimization modifies only the clock tree, this graph remains read-only. The `TimingState` connects the two structures and caches the quantities needed for fast evaluation, including clock arrivals, setup and hold slacks, TNS, WNS, area, and the proxy score.

Candidate evaluation follows the temporary apply-score-undo loop shown in Algorithm 1. For each trial edit, the solver applies the move to the `ClockTree`, updates the affected clock arrivals and related timing paths in `TimingState`, and computes the proxy-score change. The move is then either accepted or rolled back. Rollback restores both the clock-tree state and the timing cache, so rejected candidates do not affect later evaluations.

This design avoids recomputing all clock arrivals and slacks for every trial move. Instead, the solver updates only the parts of the timing state affected by the local edit, allowing many candidate moves to be evaluated within the fixed time budget.

3.3 Candidate Generation

Since the legal edit space is too large to evaluate exhaustively, candidate generation defines the local neighborhood seen by the optimizer at each iteration. A good candidate policy should include moves that are likely to improve useful skew, but it should also keep the candidate set small enough that many iterations can be completed within the time budget. We therefore explored several policies with different degrees of timing awareness and search breadth.

Algorithm 1 Overall local-search optimization flow

```
1: Build the initial ClockTree, DataPathGraph, and TimingState
2: Evaluate the baseline tree and compute its proxy score
3:  $T_{\text{cur}} \leftarrow$  baseline clock tree
4:  $T_{\text{best}} \leftarrow T_{\text{cur}}$ 
5:  $S_{\text{best}} \leftarrow S_{\text{proxy}}(T_{\text{cur}})$ 
6: while current time < time budget do
7:   Generate a candidate move set  $M$  using the candidate policy
8:    $E \leftarrow \emptyset$ 
9:   for all  $m \in M$  do
10:    Temporarily apply  $m$  to  $T_{\text{cur}}$ 
11:    Update timing and area metrics
12:    Compute  $\Delta S_{\text{proxy}}(m)$ 
13:    Store the evaluated move information in  $E$ 
14:    Undo  $m$  and restore  $T_{\text{cur}}$ 
15:   end for
16:   Select a move  $m^*$  from  $E$  using the acceptance strategy
17:   if  $m^*$  exists then
18:     Apply  $m^*$  to  $T_{\text{cur}}$ 
19:     Update TimingState for the current tree
20:     if  $S_{\text{proxy}}(T_{\text{cur}}) > S_{\text{best}}$  then
21:        $T_{\text{best}} \leftarrow T_{\text{cur}}$ 
22:        $S_{\text{best}} \leftarrow S_{\text{proxy}}(T_{\text{cur}})$ 
23:     end if
24:   end if
25: end while
26: Restore  $T_{\text{best}}$ 
27: Perform full timing evaluation
28: Write modified_clk_tree.structure
```

The simplest policy is **random generation**. It samples legal insert, remove, and resize moves from the current clock tree. This policy is not directly guided by timing violations, so many sampled moves may be weak. However, it provides broad exploration and can expose non-obvious area-recovery or resizing opportunities that violation-focused policies may miss.

The most direct timing-driven policy is **violation path generation**. It scans the data paths, selects paths with negative slack, and prioritizes the most severe violations. For a setup violation, the policy generates moves near the capture flip-flop, because delaying the capture clock increases skew and can improve setup slack. For a hold violation, it generates moves near the launch flip-flop, because delaying the launch clock decreases skew and can improve hold slack. This policy follows the useful-skew formula closely, but its neighborhood can be too narrow when the best repair is not immediately adjacent to the violating endpoint.

To address this limitation, the **upstream-window policy** expands the search region from a violating endpoint toward the root of the clock tree. Instead of considering only the endpoint edge, it considers several upstream buffers and edges. These moves may affect a larger downstream subtree and can therefore repair several related timing paths at once. The cost is that the candidate set becomes larger and each iteration spends more time evaluating moves.

The **critical-endpoint policy** uses path violations to assign a criticality value to each flip-flop. Negative setup slack contributes to the criticality of the capture flip-flop, while negative hold slack contributes to the criticality of the launch flip-flop. The policy then generates moves around the most critical flip-flops. Compared with selecting individual violating paths, this policy tries to identify endpoints that participate in many violations and may therefore produce moves with broader timing impact.

The previous timing-driven policies use different ways to locate promising clock-tree edits. The **union-pool policy** combines the action sources of violation path, upstream window, and critical endpoint into one candidate pool. To keep the evaluation cost comparable to the individual policies, it uses the same total sampling budget instead of simply increasing the number of evaluated candidates. Its purpose is therefore not to make the candidate set larger, but to make it more diverse under the same budget. Duplicate candidates are removed before evaluation.

Finally, the **sampled-union-pool policy** is a stochastic variant designed for simulated-annealing-based optimizers. Since simulated annealing evaluates a single proposal and immediately decides whether to accept it, evaluating the full union-pool candidate set at every step would not match its single-move search behavior. Instead, this policy samples only one candidate move from the union pool in each iteration. This keeps the proposal distribution diverse while preserving the low evaluation cost required by simulated annealing.

3.4 Acceptance Strategies

After candidate generation defines the visible neighborhood, the acceptance strategy determines how the search moves through that neighborhood. This decision is important because the proxy score is not a smooth objective. A move that is locally worse may still help the solver reach a better region later, while a purely greedy sequence may quickly become trapped after all immediate improvements are exhausted.

The **greedy strategy** is the most exploitative choice. It evaluates all candidates in the current set, accepts only moves with positive proxy-score change, and applies the move with the largest improvement. This strategy is stable and easy to interpret: every accepted move immediately improves the current solution according to the proxy score. However, its performance depends heavily on the candidate pool. If the pool does not contain a good improving move, greedy search cannot escape the current local optimum.

The **two-step strategy** separates timing repair and score recovery into two phases. In the first phase, the solver emphasizes a slack-focused objective, especially setup TNS and WNS. In the second phase, it returns to the full proxy score so that hold timing and area are considered together with setup timing. This strategy follows the intuition that the solver can first create timing margin and then recover area. Its weakness is that the phase boundary is fixed: area recovery may undo previous timing repairs, and timing-focused moves may create too much area overhead for the second phase to repair.

Simulated annealing [1] introduces stochastic exploration. At each step, it samples a proposal, evaluates its proxy-score change, and always accepts improving moves. A worsening move may also be accepted with a temperature-controlled probability. At high temperature, the search can move through worse intermediate states and escape local optima; at low temperature, it behaves more like greedy search. The solver still records the best-seen state separately, so temporary degradation does not directly determine the final output.

Iterated simulated annealing repeats this exploration process in several rounds. Between rounds, the solver restores the best-seen state and performs small greedy cleanup batches. The SA phases provide exploration, while the cleanup phases exploit local improvements around promising regions. This design attempts to keep the ability to escape local optima without letting the current state drift too far from a high-quality solution.

Tabu search [2] uses short-term memory instead of random temperature to guide exploration. At each step, it evaluates the candidate set and chooses the best non-tabu move. Recently used moves are marked tabu so that the solver is discouraged from cycling between the same few states. A tabu move can still be accepted by aspiration if it improves the global best score. This makes tabu search more exploratory than greedy search while still keeping each step strongly score-driven.

Overall, candidate generation and acceptance play complementary roles. Candidate policies decide which legal edits are worth looking at, while acceptance strategies decide how aggressively the solver should exploit or explore the evaluated moves. The optimizer variants evaluated in Section 4 are different combinations of these two choices.

3.5 Output Format Validation

The final solver output must be a legal `modified_clk_tree.structure` file, not only a high-scoring internal state. For this reason, the same `ClockTree` model is used for optimization and for writing the output. After the selected optimizer finishes, the solver holds the best-seen clock tree and writes it through a preorder traversal with depth information, matching the format of the input clock-tree structure.

Output validity is protected during move construction. Inserted buffers receive unique generated names, resize and insertion moves are rejected when the chosen cell does not support the required fanout, and remove moves are limited to optimizer-inserted buffers. Original contest nodes remain alive and keep their names. Search-only buffers that were removed are marked inactive and are skipped by the output traversal, so they do not appear in the final structure file.

The writer emits only active nodes and their active parent-child relationships. Therefore, each flip-flop keeps a single clock-tree parent, the tree connectivity remains well formed, and every emitted buffer has a cell type that satisfies the fanout table in the buffer library. The final write uses a temporary file followed by a rename, so the output path is replaced only after a complete structure file has been produced.

4 Experimental Results and Discussion

4.1 Using Different Candidate Policy

We first compare different candidate policies under the greedy algorithm.

Score Analysis. Overall, union pool is the best candidate policy, while random and upstream window are close behind. Violation path and critical endpoint perform worse because they do not improve both setup and hold slack sufficiently. The stronger policies have similar setup performance, but union pool gains extra score from better hold improvement and more controlled area.

Possible Reason. Greedy search always chooses the best immediately improving move. Therefore, its performance depends heavily on whether the candidate pool contains a good local edit. Violation path and critical endpoint generate narrower candidate sets, so they are more likely to miss useful upstream or area-recovery moves.

4.2 Using Different Accept Policy

We next compare accept policies using random candidates, union pool, and sampled union pool.

4.2.1 Random Candidate

With random candidates, tabu search gives the best result, while two-step optimization is the weakest. The main issue of two-step optimization is that its first stage focuses on timing repair and its second stage tries to recover area afterward. This separation is too rigid: area-recovery moves may undo previous timing repair, while timing-focused moves may leave too much area cost for the second stage to clean up. As a result, two-step optimization has weaker hold slack and area than SA, ISA, and tabu search. Its timing-first stage also does not outperform SA/ISA enough to justify the later cleanup cost. Tabu search performs better because it can accept non-improving intermediate moves, avoid recent cycles with tabu memory, and still restore the best solution found.

4.2.2 Union Pool and Sampled Union Pool

With union pool and sampled union pool, two-step optimization is still the weakest. Although union pool provides more candidates, the two fixed stages do not convert this larger search space into a better final score. SA and ISA aggressively improve setup slack, but they pay a large area penalty because many accepted moves insert or preserve extra buffers. Tabu search is more balanced: it improves timing while keeping the final area close to the baseline.

4.3 Overall Performance

Table 1 reports the average over five testcases and five seeds. Setup and Hold are S_{setup} and S_{hold} as defined in Section 2, i.e. the sum of TNS and WNS improvement for the SS and FF corners respectively; higher is better. Area is S_{area} ; higher means the optimizer reduced buffer area below the baseline. Area Ratio is the final area divided by the baseline area; lower is better (below 1.0 means area reduction). Avg. Score is the weighted total S with $\alpha = 0.5$, $\beta = 0.25$, $\gamma = 0.25$; higher is better. Std. is the standard deviation of the final scores across runs; lower indicates more consistent behavior.

Table 1: Performance comparison of different optimizers.

Optimizer	Avg. Score	Setup	Hold	Area	Area Ratio	Std.
tabu-random	1.140 722	1.276 422	1.916 255	0.093 788	0.906 212	0.235 526
isa-random	1.084 773	1.322 733	1.737 303	-0.043 676	1.043 676	0.273 375
tabu-union-pool	1.062 058	1.259 221	1.731 261	-0.001 474	1.001 474	0.274 410
isa-sampled-union-pool	1.044 759	1.521 136	1.721 173	-0.584 406	1.584 406	0.183 581
sa-sampled-union-pool	1.044 015	1.653 149	1.833 949	-0.964 191	1.964 191	0.165 838
greedy-union-pool	1.042 167	1.193 238	1.757 868	0.024 327	0.975 673	0.266 567
greedy-upstream-window	1.031 489	1.227 970	1.710 767	-0.040 751	1.040 751	0.275 511
greedy-random	1.009 059	1.104 560	1.806 695	0.020 420	0.979 580	0.214 807
sa-random	0.998 808	1.149 864	1.750 100	-0.054 596	1.054 596	0.276 991
two-step-random	0.876 061	1.148 869	1.334 986	-0.128 479	1.128 479	0.318 905
two-step-union-pool	0.859 649	1.045 729	1.405 060	-0.057 920	1.057 920	0.401 216
greedy-violation-path	0.829 324	1.032 678	1.300 464	-0.048 521	1.048 521	0.277 805
greedy-critical-endpoint	0.746 583	0.923 081	1.188 662	-0.048 491	1.048 491	0.375 666

The table shows that maximizing one timing component is not sufficient. For example, *sa-sampled-union-pool* has the highest setup component, but its area ratio is almost two times the baseline. This area penalty lowers its final score below tabu-based methods. In contrast, *tabu-random* has a lower setup component, but it achieves the best hold component and reduces area, which leads to the best overall score.

4.4 Difference Between Random and (Sampled) Union Pool on ISA and Tabu

The best strategy is tabu search with random candidates. ISA with sampled union pool improves setup slack the most, but this improvement is obtained at the cost of much larger area. In contrast, tabu search with random candidates still improves setup slack significantly, achieves the strongest hold improvement, and reduces area consumption. Therefore, its advantage comes from the best trade-off among setup, hold, and area rather than from maximizing a single component.

4.4.1 Possible Reason

ISA and tabu search are both designed to escape local optima. Random candidates provide wider exploration, while tabu memory prevents the search from repeatedly returning to recently used moves. This also explains why *tabu-random* outperforms *tabu-union-pool*: union pool is more timing-aware, but it can bias the search toward violation-related edits, while random candidates expose more non-local resize and removal opportunities. This combination works better than a purely timing-focused sampled union proposal when area must also be controlled.

5 Conclusion

In this work, we formulate timing fixing by useful skew as a clock-tree optimization problem over three reversible operations: buffer insertion, inserted buffer removal, and buffer resizing. These operations allow the optimizer to change clock arrival times at flip-flops and improve setup or hold slack without modifying the data-path logic. Since each edit may affect multiple paths and also changes area, the main challenge is not only timing repair, but the balance among setup slack, hold slack, and area cost.

Our experiments show that both candidate generation and acceptance strategy have a significant impact on the final result. Greedy search benefits from a larger candidate pool because it only accepts immediately improving moves. In contrast, simulated annealing and tabu search can explore beyond local optima, which makes random candidate generation more useful. Among all tested optimizers, tabu search with random candidates achieves the best average score. It does not produce the highest setup component, but it achieves the strongest hold improvement while also reducing area, giving the best overall trade-off. SA and ISA can aggressively improve timing, especially setup slack, but their area penalty reduces the final score. Two-step optimization follows a reasonable timing-first and area-recovery idea, but its fixed phase separation does not recover enough hold slack and area to compete with tabu search.

For future work, an adaptive strategy could select candidate policies according to the current violation pattern instead of using a fixed policy throughout the run. Another promising direction is to keep the exploration strength of tabu search while adding a stronger area-aware resize and removal polish in the late stage. This may preserve the timing gains from useful skew while further reducing unnecessary buffer area.

Author Contributions

Hong Bin Lai did nothing :) That's True, I'm very sorry. I find the branch yesterday but I see the first version submission when I'm writing. I know it's a bad excuse, and I really doesn't do anything.

Generative AI Usage Disclosure

We used generative AI tools during the development of this project. In particular, OpenAI Codex and Cursor was used extensively to assist with source-code implementation, including parser construction, clock-tree data structures, timing evaluation routines, optimization-loop implementation, test scripts, and refactoring. Approximately 80% of the submitted source code was initially generated or modified with the assistance of Codex or Cursor.

The algorithmic ideas, including the useful-skew optimization strategy, candidate generation policies, acceptance strategies, timing-area trade-off considerations, experimental design, and final optimizer selection, were designed and evaluated by the team members. All AI-generated code was manually reviewed, integrated, debugged, and tested by the team. The reported experimental results were obtained by executing our implemented solver on the test cases; no experimental data or performance numbers were generated by AI tools.

We also used ChatGPT/Codex to assist with report organization, wording refinement, and documentation polishing. The final technical content, claims, results, and conclusions were checked and approved by the team.

References

- [1] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.
- [2] Fred Glover. Tabu searchpart i. *ORSA Journal on computing*, 1(3):190–206, 1989.